EmberDB: A FHIR-Native Time-Series Storage Engine for Continuous Patient Monitoring

Abhinav Srivastava CTO, Ambra
Abhinav@ambra911.com

***Abstract*—EmberDB is a Rust time-series engine purpose-built for the continuous-monitoring workload of an ICU: high-frequency ingest of vital-sign streams, bedside point lookups, and threshold alerting on HL7 FHIR Observation data. On an identical MIMIC-IV-schema synthetic workload of 500 patients, 48-hour ICU stays, and 1.728M chartevents, EmberDB sustains 1.42M records/sec on ingest and answers patient-scoped point and latest-value lookups in single-digit microseconds, 3.9–$9.4\times$ faster on ingest and 9–$1{,}600\times$ faster on point queries than an embedded SQLite store and two purpose-built time-series engines, InfluxDB 2.7 and TimescaleDB on PostgreSQL 16. We further validate the engine on the *real*, open-access MIMIC-IV Clinical Database Demo (v2.2, $\sim$100 patients): the ingest and point/cohort-query advantages persist on real `chartevents`, while two query shapes regress at the smaller, sparser scale. EmberDB obtains this by encoding the clinical identity of an Observation in the storage key: each record carries a composite metric key of patient identifier, LOINC code, and unit, and the data is partitioned into 1-hour chunks behind a write-ahead log, so a bedside read is a prefix match with no SQL parse and no FHIR reconstruction. The engine also runs clinical pattern detection on the same data path; we benchmark four algorithms (CUSUM changepoint, seasonal decomposition, Mahalanobis anomaly scoring, and moving-window analysis). The cost of the design is analytical: EmberDB is slower on cross-patient cohort scans (TimescaleDB is competitive) and full-patient-stay reads (SQLite is faster), and less compact on disk (InfluxDB's columnar compression is $7.5\times$ denser). These analytical query shapes are outside the target workload and are reported alongside the monitoring results for context. The primary cross-system results are single-node, at a single scale, and use a synthetic schema; the real-demo run is a second, smaller-scale data point rather than full MIMIC-IV. Part of the point-query gap over the client-server baselines also reflects EmberDB's in-process library path versus their network round-trip, an axis we separate in the evaluation.**

***Index Terms*—FHIR, time-series database, patient monitoring, ICU, clinical informatics**

I. Introduction

An ICU bedside monitor samples heart rate, blood pressure, oxygen saturation, respiratory rate, and temperature at intervals that run from one second to a few minutes. Across a hospital that is millions of observations per patient-day [1]. HL7 FHIR is now the standard wrapper for these readings: each measurement is an `Observation` resource that carries a LOINC-coded type, a numeric value with units, a patient reference, and a timestamp [4].

The difficulty is not storing these observations efficiently but storing them efficiently *while preserving the clinical structure*. InfluxDB [5] and TimescaleDB [6] absorb the throughput, but neither keeps the relationship between patient, observation code, and clinical context intact at the storage layer. InfluxDB flattens each Observation into a measurement/tag/field triple. TimescaleDB keeps the relational schema but pays row-storage and SQL-parsing cost on every high-frequency point lookup, and plain PostgreSQL falls back to joins for temporal queries because it has no time-partitioned storage of its own. In all three cases the clinical structure is reconstructed at query time rather than carried through.

EmberDB is a Rust time-series storage engine built for exactly this workload: high-frequency ingest of vital-sign streams, bedside point and latest-value lookups, and threshold alerting. Its central design decision is to encode the clinical identity of an Observation in the storage key. Each record carries a composite metric name of patient ID, LOINC code, and unit, so a patient-scoped read is a prefix match on that key and nothing has to be mapped back to FHIR after ingestion. Records are bucketed into configurable time chunks, one hour by default, so a range query touches only the chunks it overlaps and retention drops whole chunks at once. A write-ahead log with chunk-level durability markers gives crash recovery without an fsync on the ingestion hot path. A set of pattern-detection algorithms runs inside the engine on the same records, so changepoint and anomaly scoring need no export step. This paper benchmarks four of those algorithms (CUSUM changepoint, seasonal decomposition, multivariate Mahalanobis anomaly detection, and moving-window analysis); a PELT segmentation method is also implemented but is not benchmarked here.

We evaluate EmberDB on a MIMIC-IV-schema synthetic `chartevents` workload and compare it against three baselines on identical data: an embedded SQLite store and two purpose-built time-series engines, InfluxDB 2.7 and TimescaleDB on PostgreSQL 16. We then repeat the four-system comparison on the *real*, open-access MIMIC-IV Clinical Database Demo to test whether the ordering survives real `chartevents`. EmberDB sustains 1.42M records/sec on ingest, 3.9–$9.4\times$ the baselines, and answers patient-scoped point and latest-value lookups in single-digit to low-tens of microseconds, 9–$1{,}600\times$ faster than the others, because each such read reduces to a hash lookup on the metric key with no SQL parse and no row reconstruction. Sections V and VI report these measurements in full.

The prefix-keyed, row-shaped design that benefits the

monitoring workload is comparatively weak on analytical queries it does not target: cross-patient cohort scans, where TimescaleDB's columnar index is competitive; full-patient-stay reads, where SQLite's single index scan is faster; and disk footprint, where InfluxDB's columnar compression is $7.5\times$ denser. These results are retained in the cross-system comparison because they are intrinsic costs of the design rather than tunable defects: an analytical query mix is out of scope, and the dimensions on which a monitoring store is slower bound the applicability of the result. Section V-A documents the configuration of every system so the comparison can be evaluated on equal terms. Sections III and IV describe the engine and its detection algorithms; Section V reports the measurements; Section VI discusses where the design performs well, where it does not, and why.

II. Background and related work

A. Clinical time-series data

Clinical monitoring streams differ from IoT or infrastructure metrics in three ways that bear directly on storage. The signals are multivariate and physiologically correlated, so heart rate, blood pressure, and SpO2 do not move independently and are usually read together for a single patient. Each value carries semantics that must survive end to end: a LOINC code, a patient reference, and a unit, none of which a bare `(time, value)` pair captures. Sampling is irregular, with the cadence set by patient acuity and clinical protocol rather than a fixed interval. The dominant access pattern in a clinical dashboard is also narrow: retrieve every vital for one patient over a shift or stay, which is a patient-scoped read rather than the cross-series aggregation that infrastructure monitoring favors.

MIMIC-IV [2] is the reference dataset for these properties, the successor to MIMIC-III [1]. Its `chartevents` table holds hundreds of millions of rows across thousands of ICU stays, keyed by an item ID that maps to a specific physiological measurement. We use its schema and published vital-sign distributions to drive the primary (synthetic) workload in this paper, and we additionally benchmark on the open-access MIMIC-IV Clinical Database Demo [3], a $\sim$100-patient subset released under the Open Data Commons Open Database License with no credentialing requirement.

B. General-purpose time-series stores

InfluxDB organizes data by measurement name with tag-based indexing, a model tuned for infrastructure monitoring. Storing FHIR Observations in it means flattening each one into a measurement/tag/field triple, which discards the structured patient/code/value relationship and reconstructs it through Flux at query time. Its TSM engine borrows the columnar, delta-and-Gorilla compression lineage of Facebook's Gorilla [7], which is why, as our results confirm, it stores the data far more compactly than a row or document format. TimescaleDB takes the opposite tack: it adds time-partitioned hypertables to PostgreSQL so the relational schema and SQL survive, at the cost of row-based storage and a SQL parse-and-plan step on the hot path for high-frequency point lookups. OpenTSDB and QuestDB are likewise purpose-built time-series stores; like InfluxDB, both treat every metric as a generic numeric stream with no first-class notion of a healthcare resource model.

C. Embedded and relational baselines

SQLite is a common embedded choice for clinical applications, and its in-process simplicity is the appeal: there is no server, no network hop, and the whole store is a single file. The cost is the absence of time-aware partitioning, so temporal queries depend on manually managed composite indexes. We include it precisely because it shares EmberDB's in-process path, which lets the comparison isolate storage-layer behavior from client-server transport (Section V-A).

D. FHIR-aware storage

The usual route to FHIR-conformant storage is a document or relational FHIR server such as HAPI FHIR [8], which stores each Observation as a JSON resource indexed for FHIR search. That preserves the full resource model but is built for transactional record-level access, not for ingesting and scanning millions of high-frequency points. The complementary route maps FHIR onto a common analytical schema such as OMOP CDM [9] for retrospective research. Neither targets the live, high-cadence monitoring path. EmberDB sits in the gap: it keeps the clinically meaningful subset of the Observation (patient, code, unit, value, time) in the storage key rather than as a reconstructed view, and optimizes for the patient-scoped read instead of generic FHIR search or batch analytics.

III. System design

A. Architecture overview

EmberDB is written in Rust and exposes both a library API and a REST interface built on `warp`. The engine has three layers: an in-memory chunk manager that holds the working set as native Rust structures, a write-ahead log that makes each write recoverable, and a JSON chunk persistence layer that flushes sealed chunks to disk. The whole engine runs in the host process, so the library API is a direct function call with no serialization or transport between the caller and the storage layer. The in-process path is intrinsic to the library API and, as Section V discusses, is a confound the evaluation controls for when comparing against client-server engines.

B. FHIR-native metric encoding

The single design decision that shapes the rest of the engine is that the clinical identity of an Observation lives in the storage key. Each FHIR Observation becomes an internal `Record` whose metric name is a composite of three fields:

```
metric_name = "{patient_id}|{loinc_code}|{unit}"
```

A heart rate for patient P1234 is stored under the key `P1234|8867-4|/min`. The encoding lets the engine answer a patient-scoped or code-scoped query by prefix matching on the key, with no secondary indexes and no join back to a FHIR resource table. The record body keeps the timestamp, the numeric value, the FHIR resource type, and a small context map for fields such as device identifier, so the record remains

a faithful projection of the source Observation rather than a lossy summary.

Multi-component observations are decomposed into records that share a key prefix. Blood pressure is the canonical case: its systolic and diastolic components become two records under sibling LOINC codes for the same patient. The FHIR component structure is preserved, and each component is then a time series in its own right that the detection algorithms can score independently.

C. Time-partitioned chunks

Records live inside `TimeChunk` structures, each spanning a configurable duration of one hour by default. A chunk holds three things: a `HashMap<String, Vec<Record>>` that maps each metric key to a time-ordered vector of its records, a resource-type index that maps a FHIR resource type to the set of metric keys it contains, and a metadata block (creation time, last-access time, record count, and a dirty flag). The per-key vector keeps records in arrival order, which for a monotonic monitoring stream is also timestamp order, so a latest-value read is the tail of the vector and a range read is a bounded scan within it.

The chunk that owns a timestamp is found by flooring the timestamp to the nearest multiple of the chunk duration, a single integer operation, so chunk identification is $O(1)$ and needs no lookup structure. A range query computes the first and last chunk it overlaps and visits only those, which bounds the work to the queried window rather than the whole store. Retention is a chunk-granular operation: dropping a day of history is dropping twenty-four chunk files, with no row-level deletes.

D. Query execution model

A read resolves in two steps. The engine first selects the chunks the time window overlaps, then within each chunk it does one `HashMap` lookup on the metric key and a linear filter over that key's vector for the requested range. A single-patient single-vital query is therefore one hash lookup plus a short scan, which is why it runs in microseconds. The same model has a structural consequence the evaluation makes concrete: a query that spans many keys, such as one vital across a 500-patient cohort, issues one keyed lookup per patient, because the cohort is not itself a key. A latest-value read scans the resident chunks for the key and keeps the newest tail. These per-key access patterns favor the patient-scoped reads the engine targets and are comparatively costly for cross-key cohort scans, a tradeoff we return to in Section VI.

E. Write-ahead log and durability

Durability is layered so that the cost is paid at flush time rather than on every write. With persistence enabled, a record is appended to the WAL before it lands in its in-memory chunk; the WAL entry is the JSON-serialized record behind a 4-byte big-endian length prefix, and batches of more than a hundred records are written in a single append to amortize the syscall. A chunk is sealed and flushed to disk once it crosses 10,000 records or 1 MB, written to a temporary file, `fsync`ed, and atomically renamed into place. When a chunk is durable on disk, the WAL entries it covers are marked durable and become eligible for truncation, so the log does not grow without bound. On startup the engine loads the persisted chunks and replays whatever WAL entries remain past the last durable chunk, which reconstructs the records that had not yet been flushed when the process stopped. The synchronous per-record `fsync` path is the strict-durability mode and incurs a per-record synchronization cost; the comparative ingestion numbers use the in-memory ingest path, a choice we hold equal across systems and document in Section V-A.

F. MIMIC-IV data loader

A loader module maps MIMIC-IV `chartevents` item IDs to FHIR LOINC codes (itemid 220045 $\rightarrow$ LOINC 8867-4 for heart rate, and so on) and builds `Record` structures directly from the rows. The conversion writes the composite key and value into the record without first materializing an intermediate FHIR JSON document, so the ingestion path carries no per-record JSON parse on the way in. The real-demo run (Section V-E) reuses this item-ID-to-LOINC mapping but parses the demo CSV's actual 11-column, ISO-datetime schema, which differs from the integer-timestamp layout the synthetic generator emits.

IV. Pattern detection

EmberDB provides built-in pattern detection algorithms, configured via a TOML file and operating directly on stored records. We describe five algorithms below and report measured latency for four of them in Table IV; the PELT segmentation method is implemented but, as it has not yet been benchmarked under the same harness, is omitted from the latency table and excluded from the head count of benchmarked algorithms.

A. CUSUM changepoint detection

The Cumulative Sum (CUSUM) algorithm maintains running sums S^+ and S^- tracking positive and negative deviations from the series mean:

$$S_i^+ = \max(0,\ S_{i-1}^+ + (x_i - \mu - k))$$
$$S_i^- = \max(0,\ S_{i-1}^- + (\mu - k - x_i))$$

where $k = 0.5\sigma$ is the sensitivity parameter. A changepoint is declared when either sum exceeds the threshold $h = \theta\sigma$, where θ is configurable (default 2.0).

B. PELT segmentation

The Pruned Exact Linear Time (PELT) algorithm finds the optimal segmentation by minimizing the penalized cost:

$$\sum_{j=1}^{m+1} \mathcal{C}(y_{\tau_{j-1}+1:\tau_j}) + m \cdot \beta$$

where $\mathcal{C}$ is the Gaussian negative log-likelihood cost for each segment and β is the penalty term. Changepoints with magnitude below $\theta\sigma$ are pruned. This method is implemented in the engine but is not included in the latency benchmark of Table IV.

C. Seasonal decomposition

Time series are decomposed into trend T_t, seasonal S_t, and residual R_t components using moving-average smoothing. Both additive ($Y_t = T_t + S_t + R_t$) and multiplicative ($Y_t = T_t \times S_t \times R_t$) models are supported.

D. Mahalanobis anomaly detection

For multivariate anomaly detection across correlated vitals, EmberDB computes the Mahalanobis distance:

$$D_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

Points exceeding a configurable threshold (default 3.0) are flagged as outliers. Metric groups can be specified explicitly or auto-detected via Pearson correlation.

E. Moving window analysis

A sliding window computes per-window statistics (trend slope, volatility, or range) and flags windows whose statistic deviates from the population by more than θ standard deviations.

V. Evaluation

We evaluate EmberDB with four benchmark suites: a standalone characterization of the engine in isolation, a focused comparison against an embedded SQLite store, a four-system comparison on a MIMIC-IV-schema synthetic workload against three baselines (SQLite, TimescaleDB, and InfluxDB), and a final four-system comparison on the *real* open-access MIMIC-IV Clinical Database Demo against the same three baselines. All experiments ran on a single Apple Silicon machine under macOS, with EmberDB compiled in release mode (`--release`). The benchmark drivers are committed in `benches/`, and every reported figure is recorded in a CSV artifact (`head_to_head_results.csv` for the synthetic four-system comparison and `mimic_demo_results.csv` for the real-demo run).

A. Baseline methodology

Whether a cross-system table is fair depends on how each system was configured, so we document the setup of all four before reporting any numbers. The guiding rule was to hold the workload and the measured operations identical and to give each engine the configuration its own vendor recommends for bulk time-series use, rather than the system's out-of-the-box default.

Shared workload. Every system ingests the same 1,728,000-record dataset, generated once from a fixed seed (42) by the same Rust generator and replayed into each store. The InfluxDB and TimescaleDB drivers and the EmberDB/SQLite driver call byte-for-byte the same generation routine with the same RNG stream, so the four systems see the same patients, codes, timestamps, and values. Every system answers the same four clinical query shapes against the same target patient (subject 10050) and the same time windows: a single-vital one-hour point read, a full-patient-stay scan, a one-hour single-vital cohort scan across all 500 patients, and a latest-value read. The query semantics, not just the names, are matched: each returns the same logical result set on each engine.

EmberDB. Records are ingested through the library API in in-memory ingest mode, the configuration behind the comparative throughput numbers; the chunk duration is one hour and the chunk flush threshold is 10,000 records or 1 MB. The on-disk storage figure is measured separately by `ember_storage_probe.rs`, which ingests the same 1.728M records, enables persistence, and flushes once, so the storage row reflects the steady-state JSON chunk format rather than the in-memory size. Queries are direct library calls.

SQLite. We use a bundled SQLite in WAL journal mode with `PRAGMA synchronous=NORMAL`, the standard embedded configuration that preserves durability while reducing fsync frequency. The schema is a single `chartevents` table with composite and single-column indexes on `subject_id`, `itemid`, `charttime`, and `(subject_id, itemid)`, so every query shape has an index available. Ingestion runs inside one transaction. Queries use prepared, cached statements to keep statement compilation off the per-iteration timer.

TimescaleDB. We run TimescaleDB on PostgreSQL 16 in Docker. The table is a hypertable on `charttime` with a one-hour `chunk_time_interval`, matching EmberDB's one-hour chunk so the two partition the time axis at the same granularity. Bulk load uses `COPY FROM STDIN`, the recommended fast ingest path. Indexes are created *after* the load, the standard bulk-load order: a composite `(subject_id, itemid, charttime DESC)` index for point and latest-value queries and an `(itemid, charttime)` index for the cohort scan, followed by `ANALYZE`. Queries use prepared statements over a local socket.

InfluxDB. We run InfluxDB 2.7 in Docker. Writes use the line protocol over HTTP in 10,000-point batches, with the patient and LOINC code as tags and the reading as the field, the canonical InfluxDB shape for this data. The bucket is cleared before the run so the storage measurement reflects this run alone, and we allow a flush window before measuring on-disk size so the WAL has drained into the columnar TSM engine. Queries are Flux programs sent over HTTP.

What is held equal and what is not. Table I summarizes the controlled and uncontrolled variables. The dataset, the four query semantics, the host, and the use of each vendor's recommended bulk path are held equal. Two variables are deliberately not equalized because they are intrinsic to the systems being compared. First, durability granularity differs by design: EmberDB and SQLite ingest in a fast batched mode, TimescaleDB commits through PostgreSQL's WAL, and InfluxDB flushes asynchronously, so the ingestion comparison is between each engine's recommended bulk mode, not a forced common fsync policy. Second, the client path differs: an EmberDB or SQLite query is an in-process call, a TimescaleDB query crosses a local socket and is parsed and planned as SQL, and an InfluxDB query is an HTTP round-trip whose Flux body is compiled on each invocation. We did not instrument the engines to subtract the transport and compilation component, so the largest query multiples,

the InfluxDB ones in particular, mix storage-engine cost with client-path cost. We note this throughout and interpret those multiples as the embedded-versus-client-server axis rather than as pure storage-layer differences. Because SQLite shares EmberDB's in-process path, the SQLite comparison isolates the storage-layer component of the gap.

TABLE I
CONTROLLED AND UNCONTROLLED VARIABLES IN THE CROSS-SYSTEM COMPARISON.

Variable	Status
Dataset (seed 42, 1.728M rec)	Held equal (identical generator)
Query shapes and semantics	Held equal (same patient/windows)
Host and OS	Held equal (one Apple Silicon host)
Index coverage	Each engine indexed for all shapes
Ingest path	Each engine's recommended bulk mode
Durability granularity	Not equalized (intrinsic to engine)
Client path (in-proc vs. network)	Not equalized (intrinsic to engine)

Measurement limits. The harness reports the arithmetic mean latency over a fixed repetition count per query (200 for EmberDB, SQLite, and TimescaleDB; 10 to 50 for the slower InfluxDB queries, to bound wall-clock time). It does not record per-iteration samples, so we report means only, with no median, p99, or dispersion measure, and we apply no explicit warmup-discard step. The microsecond-scale gaps between EmberDB and SQLite on point queries are therefore order-of-magnitude indicators rather than statistically tested differences, and the lower InfluxDB iteration count makes its means noisier. An equal, larger iteration budget with warmup discard and full latency distributions would harden these measurements.

B. Standalone performance

1) Write throughput: Table II shows write throughput across batch sizes with persistence disabled (memory-optimized mode).

TABLE II
EMBERDB WRITE THROUGHPUT (MEMORY MODE)

Records	Throughput (rec/s)
1,000	2,689,372
10,000	1,467,926
50,000	2,582,106
100,000	3,238,495
500,000	3,297,006

2) Query latency: Table III reports average query latency over 100K ingested records (100 patients, single metric).

TABLE III
EMBERDB QUERY LATENCY (μS)

Query Type	Avg Latency (μs)
Point (narrow range)	0.1
Latest value	0.9
Patient + code	126.2
Full patient (prefix)	13.6
Time range (50K span)	61.1

3) Pattern detection: Table IV reports per-invocation latency for each detection algorithm on 2,000 records.

TABLE IV
PATTERN DETECTION LATENCY (μS)

Algorithm	Avg Latency (μs)
CUSUM changepoint	598.0
Seasonal decomposition	5,167.5
Moving window	423.5
Mahalanobis (multivariate)	251.5

4) WAL performance: With persistence enabled (fsync per record), EmberDB sustains 124 records/s write throughput due to disk synchronization overhead. Recovery from WAL (10,000 records) completes in 0.008 seconds.

5) Storage overhead: At 100,000 records with full persistence, EmberDB uses 149.6 bytes per record on disk (14.96 MB total) due to JSON serialization overhead. This standalone figure is measured on a uniform 100K-record single-metric dataset by `benches/eval.rs`. The cross-system comparison below (Table VII) reports a higher 201.4 bytes/record for the same JSON chunk format, because it is measured by a separate probe (`benches/ember_storage_probe.rs`) on the full 1.728M-record MIMIC workload, which spans 500 patients and 6 vitals across many one-hour chunks. The larger figure reflects the per-chunk and per-metric-key overhead that grows with the number of distinct `patient|code|unit` keys and time chunks, which the single-metric micro-benchmark does not exercise; the 201.4 B/rec measurement is the one to use for the cross-system comparison.

C. Comparative analysis: EmberDB vs. SQLite

1) Write throughput: Table V compares write throughput. EmberDB's in-memory mode avoids disk synchronization overhead, while SQLite uses WAL mode with `PRAGMA synchronous=NORMAL`.

TABLE V
WRITE THROUGHPUT COMPARISON (REC/S)

Records	EmberDB	SQLite	Ratio
1,000	738,257	246,038	3.0×
10,000	1,258,139	247,915	5.1×
50,000	2,145,535	282,572	7.6×
100,000	2,197,044	263,857	8.3×
500,000	2,170,299	254,397	8.5×

2) Query latency: On a uniform 100K-record microbenchmark, the gap over SQLite is widest where the workload is purely patient-scoped: a point lookup runs in 0.1 μs vs. 764.5 μs (8,890×) and a latest-value read in 0.9 μs vs. 857.4 μs (953×). It narrows on scans, to 12.8× on a time range and 7.1× on a full-patient read, where both engines touch more records. The sub-microsecond point-query figure (0.1 μs) is specific to this uniform standalone setting; on the more realistic MIMIC workload below the same query shape costs 3.3 μs, and that single-digit-microsecond figure, not the

sub-microsecond one, is the appropriate point of comparison against the purpose-built engines. The MIMIC results below show the same shape against those engines.

D. MIMIC-IV-schema synthetic workload

We generated synthetic, MIMIC-IV-shaped chartevents for 500 patients, each with a 48-hour ICU stay and 6 vital signs sampled every 5 minutes (576 measurements per vital per patient). These rows follow the MIMIC-IV `chartevents` schema and item-ID-to-LOINC mapping but are drawn from published ICU distributions, not from any real patient record. Vital sign distributions were calibrated to published ICU ranges: HR $\mathcal{N}(84, 17)$, SBP $\mathcal{N}(121, 23)$, DBP $\mathcal{N}(70, 14)$, SpO2 $\mathcal{N}(96.8, 2.8)$, RR $\mathcal{N}(18, 4)$, Temp $\mathcal{N}(98.6, 1.2)$ °F. Each chartevent was mapped to a FHIR Observation via LOINC code and ingested into both EmberDB and SQLite.

1) Ingestion: The dataset contains 1,728,000 chartevents (500 patients $\times$ 6 vitals $\times$ 576 measurements). EmberDB ingested the FHIR-converted records at 1,424,089 records/sec (1.21s total). On the same data, SQLite reached 151,113 records/sec, TimescaleDB 363,352 records/sec via `COPY`, and InfluxDB 301,453 records/sec via the line protocol. EmberDB is $9.4\times$ faster than SQLite and $3.9\times$ faster than TimescaleDB, the fastest of the purpose-built baselines on ingestion. This 1.42M records/sec is the comparative figure against which the 3.9–$9.4\times$ ratios are computed. It is lower than the 3.2M records/sec of the standalone memory-mode micro-benchmark in Table II because that micro-benchmark runs with persistence disabled and no baseline alongside it; the two figures describe different configurations and should not be conflated.

2) Query performance: Table VI reports query latency for the same four query shapes on each system.

TABLE VI
MIMIC-IV QUERY LATENCY (μS), EMBERDB VS. SQLITE

Query	EmberDB	SQLite	Speedup
Single vital (1h)	3.3	31.0	9.4×
Full patient stay	955.9	624.8	0.65×
Cohort vital (1h)	817.1	18,524.6	22.7×
Latest vital	28.1	80.5	2.9×

3) Cross-system comparison against purpose-built engines: Table VII places all four systems side by side on the MIMIC workload: write throughput, the four query latencies, and on-disk storage. Lower is better for every row except ingestion.

EmberDB is fastest on ingestion and on both the single-vital and latest-vital lookups, by one to three orders of magnitude over the purpose-built engines, because each of those reads reduces to a hash lookup on the metric key plus a short in-chunk scan, with no SQL parse and no row reconstruction. The single-vital lookup runs about $160\times$ faster than TimescaleDB and over $1{,}600\times$ faster than InfluxDB; the latest-value lookup is $42\times$ and $150\times$ faster, respectively. These are the operations a bedside monitor and a threshold-alert pipeline issue continuously, and they constitute the engine's target workload. The multiples against the server engines should be read with the client-path caveat from Section V-A: the TimescaleDB and InfluxDB figures include socket or HTTP transport and, for InfluxDB, Flux compilation on every iteration, so part of the gap is the embedded-versus-client-server axis and not the storage engine alone. The SQLite column, which shares EmberDB's in-process path, isolates the storage-layer comparison, where the single-vital gap is a more modest $9.4\times$.

TABLE VII
CROSS-SYSTEM COMPARISON ON THE MIMIC-IV-SCHEMA SYNTHETIC WORKLOAD (1.728M RECORDS). INGESTION IN RECORDS/SEC (HIGHER IS BETTER); QUERY LATENCY IN μS AND STORAGE IN BYTES/RECORD (LOWER IS BETTER).

Metric	EmberDB	SQLite	TimescaleDB	InfluxDB
Ingest (rec/s)	**1,424,089**	151,113	363,352	301,453
Single vital (1h)	**3.3**	31.0	531.3	5,383
Full patient stay	955.9	**624.8**	3,078	13,167
Cohort vital (1h)	817.1	18,525	**751.8**	12,073
Latest vital	**28.1**	80.5	1,178	4,226
Storage (B/rec)	201.4	103.1	129.3	**26.8**

The table also shows where EmberDB is slower. These cases arise from the same prefix-keyed design that accelerates the point queries and fall on analytical query shapes the engine does not target. TimescaleDB is faster on the cohort scan (752 μs against EmberDB's 817 μs) because its `(itemid, charttime)` index answers the whole one-hour window for one vital in a single indexed range scan, whereas the cohort is not a key in EmberDB, so the engine issues 500 separate per-patient keyed lookups and concatenates them. SQLite is faster on the full-patient-stay query for the mirror-image reason, discussed below. InfluxDB is slowest on every query in this point/range mix, which fits its tag-cardinality model poorly, but the mix omits the time-bucketed downsampling and multi-tag aggregation that InfluxDB and TimescaleDB are tuned for, so it understates both server engines on the analytical work they are built for. InfluxDB also stores the data far more compactly: its TSM columnar compression reaches 26.8 bytes/record against EmberDB's 201.4, a $7.5\times$ gap attributable to the JSON chunk format.

E. Real MIMIC-IV demo workload

To test whether the synthetic ordering survives real data, we repeated the four-system comparison on the *real*, open-access MIMIC-IV Clinical Database Demo, version 2.2 [3], downloaded from PhysioNet[1] under the Open Data Commons Open Database License (no credentialing required). The demo is a $\sim$100-patient subset: its `icu/chartevents.csv` holds 668,862 rows across 100 distinct `subject_id`s and 140 ICU stays, of which 78,441 rows map to the loader's known LOINC vitals (all with a numeric `valuenum`). We verified the SHA-256 of the downloaded `chartevents.csv.gz` against the archive's bundled `SHA256SUMS.txt` before use. All four systems ingest and query the *same* 78,441 real rows, with the same four query shapes the synthetic run uses, over windows derived from the busiest real series (patient 10003400, heart rate). This is deliberately a smaller, sparser workload than

[1] https://physionet.org/content/mimic-iv-demo/2.2/

the 1.728M-row synthetic run: the demo charts vitals roughly hourly rather than every five minutes, and 78k mapped rows is about 22× fewer than the synthetic dataset, so all absolute throughputs are lower by design. The real demo CSV also uses an 11-column, ISO-datetime schema rather than the integer-timestamp layout the synthetic generator emits; the real-data driver (`benches/mimic_real_bench.rs`) parses that schema and reuses the production item-ID-to-LOINC mapping unchanged.

TABLE VIII
CROSS-SYSTEM COMPARISON ON THE *real* MIMIC-IV DEMO (78,441 MAPPED VITAL-SIGN ROWS, ∼100 PATIENTS). INGESTION IN RECORDS/SEC (HIGHER IS BETTER); QUERY LATENCY IN μS (LOWER IS BETTER).

Metric	EmberDB	SQLite	TimescaleDB	InfluxDB
Ingest (rec/s)	**1,944,141**	236,602	158,006	328,673
Single vital (1h)	**0.5**	142.6	554.7	4,758
Cohort vital (1h)	**155.9**	2,760	427.9	4,007
Full patient stay	4,334	**1,687**	2,151	14,546
Latest vital	7,538	**165.0**	1,768	4,758

Table VIII reports the result. The synthetic ordering is preserved on real data: EmberDB is the fastest of the four systems on ingestion and on both the point and cohort queries. It ingests the real rows at 1.94M records/sec, 8.2× faster than SQLite and the fastest of the four; the single-vital point query runs in 0.5 μs, roughly 285× faster than SQLite, a wider margin than on the synthetic run because the demo is sparse and a one-hour window holds about one sample, so EmberDB answers from a hash lookup while every other engine pays a fixed per-query overhead; and the one-hour cohort scan is fastest at 155.9 μs. The advantage on the operations a bedside monitor and an alert pipeline issue therefore persists on real `chartevents`.

Two query shapes flip the other way on real data, and both trace to the same cause. The MIMIC timestamps are de-identified by shifting each patient into a future window, so the real `charttimes` span roughly ninety calendar years (2110–2201) rather than the contiguous 48-hour stays of the synthetic generator. EmberDB's one-hour chunks are therefore scattered across an enormous key space. The `latest_vital` query, faster than SQLite on synthetic data, becomes slower (7,538 μs against SQLite's 165 μs) because `get_latest` scans every chunk for the metric and there are now thousands of sparsely populated chunks, while SQLite answers from its `(subject_id, itemid, charttime DESC)` index. The `full_patient_stay` query is slower for the same chunk-scan reason; it was already slower than SQLite on synthetic data, and the wide timestamp spread widens the gap. These design weaknesses surface only under realistically spread timestamps, and a chunk index keyed for newest-first lookup would address the `latest_vital` regression directly. EmberDB storage on the demo is again reported in the in-memory/no-persistence mode the synthetic harness uses, so on-disk footprint is not a valid EmberDB comparison on either run and is omitted from Table VIII; the demo result is a correctness-and-ordering check on real data, not a storage measurement.

VI. DISCUSSION

FHIR-native advantages. By encoding patient ID and LOINC code directly into the metric key, EmberDB removes the need for secondary indexes or joins when retrieving patient-scoped observations. The design fits clinical dashboard queries that retrieve all vitals for a single patient over a shift or stay.

Pattern detection at the storage layer. Embedding change-point detection and anomaly scoring directly in the storage engine avoids data export overhead. Real-time alerting pipelines can operate on the same data path as persistence.

Where EmberDB underperforms, and why. This section examines the three dimensions on which EmberDB underperforms a baseline. Each traces to a specific design decision rather than to a poorly chosen tunable, and each is the cost of a tradeoff that also yields one of the advantages above.

The largest gap is storage footprint. EmberDB writes 201.4 bytes/record on disk, against 129.3 for TimescaleDB, 103.1 for SQLite, and 26.8 for InfluxDB, whose TSM engine is 7.5× more compact. The cause is the chunk format. EmberDB persists each chunk as a JSON document in which every record repeats its field names and serializes its timestamp and value as text, and the composite key is stored per record. InfluxDB does the opposite: it stores each series as a column and applies delta encoding to timestamps and Gorilla-style compression to values, so repeated structure adds negligible per-record overhead. The chunk format that makes EmberDB straightforward to load without parsing is also space-inefficient on disk. Of the three gaps, this is the one a columnar chunk encoding would address most directly, closing most of the difference.

The full-patient-stay query is the second gap: SQLite returns a patient's whole 48-hour stay in 625 μs against EmberDB's 956 μs. The reason is in EmberDB's query model. A stay spans all six vitals, but each vital is a distinct metric key, so retrieving the stay means six separate keyed range queries whose results are then concatenated, each one re-scanning the relevant chunks. SQLite answers the same request as one index scan over `(subject_id, charttime)` that returns every vital interleaved. The prefix key that makes a single-vital lookup inexpensive turns a multi-vital read into several lookups, and on a six-vital stay the constant factor of issuing and merging six queries overtakes SQLite's single scan. A multi-metric range API that walks all keys sharing a patient prefix in one pass over each chunk would remove the per-vital repetition; the data layout already supports it, since the keys are co-resident in the same chunk.

The cohort scan is the third gap, and the sharpest instance of the same effect. TimescaleDB answers one vital across all 500 patients for a one-hour window in 752 μs; EmberDB takes 817 μs. A cohort is not a key in EmberDB, so the engine issues 500 per-patient keyed lookups and concatenates them, while TimescaleDB serves the whole window from its `(itemid, charttime)` index in a single indexed range scan. EmberDB remains more than 20× faster than SQLite

here because SQLite's row store touches every row in the window, but against a purpose-built columnar index on the cohort dimension the per-key model is poorly matched. A secondary index keyed by code rather than patient would make the cohort scan a single traversal, at the cost of a second index to maintain on write.

All three gaps share a root: the storage key is patient-first and the on-disk format is row-shaped JSON. This combination accelerates patient-scoped point queries and ingestion while penalizing cross-patient scans and disk footprint. A columnar chunk format with delta and value compression, a multi-metric range API, and an optional code-keyed secondary index would address the three gaps directly, and each is compatible with the existing chunk layout.

Other limitations. The single-writer architecture limits concurrent write throughput. The pattern-detection algorithms use simplified implementations; a production deployment would need validated statistical libraries.

Scope of the evaluation. The primary cross-system comparison is single-node, at a single scale, and uses a synthetic schema: one 1.728M-record point at one cardinality on one Apple Silicon host, generated from published ICU vital-sign distributions rather than real patient records. The real MIMIC-IV demo run (Section V-E) removes the synthetic-data caveat for the ingest and point/cohort queries, where the ordering held on real `chartevents`, and it adds a second data point at a smaller, sparser scale. It does not remove the single-host caveat, and it is the $\sim$100-patient open demo rather than full MIMIC-IV, which is roughly three orders of magnitude larger; the demo also exposed two regressions (`latest_vital` and `full_patient_stay`) that the contiguous synthetic timestamps had hidden, so the demo strengthens the ingest/-point claims while sharpening the known analytical-query weaknesses. All four systems ran on one machine, so the numbers describe single-host engine behavior and say nothing about clustered or networked deployment. As Section V-A sets out, the systems are also exercised through different client paths, so the query-latency multiples mix storage-engine cost with transport and query-compilation overhead that we did not separate numerically. Finally, the point/range query mix favors a prefix-keyed store: it omits the analytical scans (time-bucketed downsampling, multi-tag aggregation, gap-fill) that InfluxDB and TimescaleDB are built for, so the current mix favors a prefix-keyed store such as EmberDB. We therefore present the synthetic ordering as a hypothesis that the demo run partially confirms; a second synthetic scale and the missing analytical query shapes would further de-risk it, and the absolute storage figures would shift with a columnar format.

Future work. Planned improvements include columnar chunk encoding (e.g., Parquet), concurrent chunk writers with sharded locking, integration with FHIR Subscription for push-based alerts, and support for FHIR Bulk Data export. Validating on full MIMIC-IV (rather than the $\sim$100-patient open demo) requires PhysioNet credentialed access and a data use agreement and is left for future work.

VII. Conclusion

EmberDB combines high ingest throughput with native clinical semantics. Encoding patient identity and LOINC codes into the storage key, partitioning the data into time-aligned chunks, and integrating the pattern-detection algorithms directly yields a system that handles continuous patient monitoring while retaining time-series performance. On the MIMIC-IV-schema synthetic workload it ingests 3.9–$9.4\times$ faster than SQLite, TimescaleDB, and InfluxDB and answers patient-scoped point queries in single-digit microseconds, while preserving FHIR structure with no post-hoc transformation, and the ingest and point/cohort-query advantages reproduce on the real, open-access MIMIC-IV demo. EmberDB is not faster on every operation: TimescaleDB is competitive on cohort scans and InfluxDB stores the data far more compactly, both of which point to the columnar chunk format as the next step, and the real-demo run further exposed a latest-value and full-stay regression under realistically spread timestamps. These results come from one host, measured as means without latency distributions and through engine-specific client paths, so we present the ordering as a hypothesis that the real-demo run partially confirms; hardening it calls for a second synthetic scale, validation on full credentialed MIMIC-IV, the analytical query shapes the current mix omits, and full latency distributions over an equal iteration budget.

Acknowledgment

The author used a generative AI assistant solely for language editing (grammar and sentence structure). All research, implementation, benchmarking, analysis, and conclusions are the author's own original work; the author reviewed and verified the entire manuscript and takes full responsibility for its content.

References

[1] A. E. W. Johnson, T. J. Pollard, L. Shen, et al., "MIMIC-III, a freely accessible critical care database," *Scientific Data*, vol. 3, p. 160035, 2016.

[2] A. E. W. Johnson, L. Bulgarelli, L. Shen, et al., "MIMIC-IV, a freely accessible electronic health record dataset," *Scientific Data*, vol. 10, p. 1, 2023.

[3] A. Johnson, L. Bulgarelli, T. Pollard, S. Horng, L. A. Celi, and R. Mark, "MIMIC-IV Clinical Database Demo (version 2.2)," *PhysioNet*, 2023. [Online]. Available: https://doi.org/10.13026/dp1f-ex47

[4] HL7 International, "FHIR Release 4: Observation Resource," https://hl7.org/fhir/R4/observation.html, 2019.

[5] InfluxData, "InfluxDB: Open Source Time Series Database," https://www.influxdata.com/, 2024.

[6] Timescale Inc., "TimescaleDB: An Open-Source Time-Series SQL Database," https://www.timescale.com/, 2024.

[7] T. Pelkonen, S. Franklin, J. Teller, et al., "Gorilla: A fast, scalable, in-memory time series database," *Proc. VLDB Endowment*, vol. 8, no. 12, pp. 1816–1827, 2015.

[8] Smile CDR Inc., "HAPI FHIR: The Open Source FHIR API for Java," https://hapifhir.io/, 2024.

[9] G. Hripcsak, J. D. Duke, N. H. Shah, et al., "Observational Health Data Sciences and Informatics (OHDSI): Opportunities for observational researchers," *Stud. Health Technol. Inform.*, vol. 216, pp. 574–578, 2015.